Unix/Linux System Programming & OS Concepts Cheat Sheet

1. Basic Unix Shell Commands

File and Directory Operations

```bash
# List files and directories
ls              # Basic listing
ls -l           # Detailed listing
ls -la          # Include hidden files with details
ls -R           # Recursive listing

# Navigate directories
pwd             # Print working directory
cd /path/to/dir     # Change directory
cd ..           # Go to parent directory
cd ~            # Go to home directory

# File operations
cp source dest      # Copy file
mv source dest      # Move/rename file
rm filename         # Remove file
rm -r dirname       # Remove directory recursively
touch filename      # Create empty file or update timestamp
```

File Content Operations

```bash
cat filename        # Display file content
less filename       # View file page by page
head -n 10 file     # Show first 10 lines
tail -n 10 file     # Show last 10 lines
grep "pattern" file  # Search for pattern in file
wc -l filename      # Count lines in file
chmod 755 filename  # Change file permissions
```

2. Unix System Calls

Process Control System Calls

fork() - Create Child Process

```c
```

```c
#include <unistd.h>
#include <sys/types.h>

pid_t fork(void);

// Example usage
pid_t pid = fork();
if (pid == 0) {
    // Child process code
    printf("This is child process\n");
} else if (pid > 0) {
    // Parent process code
    printf("This is parent process, child PID: %d\n", pid);
} else {
    // Fork failed
    perror("fork failed");
}
```

exec() Family - Execute New Program

```c
#include <unistd.h>

int execl(const char *path, const char *arg0, ..., NULL);
int execlp(const char *file, const char *arg0, ..., NULL);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);

// Example usage
execl("/bin/ls", "ls", "-l", NULL);
// or
char *args[] = {"ls", "-l", NULL};
execv("/bin/ls", args);
```

getpid() - Get Process ID

```c
```

```c
#include <unistd.h>
#include <sys/types.h>

pid_t getpid(void);  // Current process ID
pid_t getppid(void); // Parent process ID

// Example
printf("Current PID: %d\n", getpid());
printf("Parent PID: %d\n", getppid());
```

exit() - Terminate Process

```c
#include <stdlib.h>
#include <unistd.h>

void exit(int status);   // Normal exit
void _exit(int status);  // Immediate exit

// Example
exit(0);   // Success
exit(1);   // Error
```

wait() - Wait for Child Process

```c
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);

// Example
int status;
pid_t child_pid = wait(&status);
if (WIFEXITED(status)) {
    printf("Child exited with status: %d\n", WEXITSTATUS(status));
}
```

File System Calls

close() - Close File Descriptor

```c
```

```c
#include <unistd.h>

int close(int fd);

// Example
int fd = open("file.txt", O_RDONLY);
// ... use file
close(fd);
```

stat() - Get File Information

```c
#include <sys/stat.h>
#include <sys/types.h>

int stat(const char *path, struct stat *buf);
int fstat(int fd, struct stat *buf);

// Example
struct stat file_stat;
if (stat("file.txt", &file_stat) == 0) {
    printf("File size: %ld bytes\n", file_stat.st_size);
    printf("File mode: %o\n", file_stat.st_mode);
}
```

opendir() and readdir() - Directory Operations

```c
#include <dirent.h>
#include <sys/types.h>

DIR *opendir(const char *name);
struct dirent *readdir(DIR *dirp);
int closedir(DIR *dirp);

// Example
DIR *dir = opendir("/path/to/directory");
struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    printf("File: %s\n", entry->d_name);
}
closedir(dir);
```

3. Shell Programming Basics

Basic Shell Script Structure

bash

```bash
#!/bin/bash
# This is a comment

# Variables
name="John"
age=25

# Output
echo "Name: $name"
echo "Age: $age"

# Command substitution
current_date=$(date)
echo "Current date: $current_date"
```

Control Structures

bash

```bash
# If statement
if [ $age -gt 18 ]; then
  echo "Adult"
elif [ $age -eq 18 ]; then
  echo "Just turned adult"
else
  echo "Minor"
fi

# For loop
for i in {1..5}; do
  echo "Number: $i"
done

# While loop
counter=1
while [ $counter -le 5 ]; do
  echo "Counter: $counter"
  ((counter++))
done
```

4. Process and Thread Creation

Process Creation Example

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child process executing...\n");
        execl("/bin/echo", "echo", "Hello from child", NULL);
    } else if (pid > 0) {
        // Parent process
        printf("Parent waiting for child...\n");
        wait(NULL);
        printf("Child completed\n");
    }

    return 0;
}
```

Thread Creation (POSIX Threads)

```c
```

```c
#include <pthread.h>
#include <stdio.h>

void* thread_function(void* arg) {
    printf("Thread executing: %s\n", (char*)arg);
    return NULL;
}

int main() {
    pthread_t thread;
    char* message = "Hello from thread";

    // Create thread
    pthread_create(&thread, NULL, thread_function, message);

    // Wait for thread to complete
    pthread_join(thread, NULL);

    return 0;
}
```

5. CPU Scheduling Algorithms

First Come First Serve (FCFS)

c

```c
void fcfs_scheduling(int processes[], int n, int burst_time[]) {
    int waiting_time[n], turnaround_time[n];

    waiting_time[0] = 0;
    for (int i = 1; i < n; i++) {
        waiting_time[i] = waiting_time[i-1] + burst_time[i-1];
    }

    for (int i = 0; i < n; i++) {
        turnaround_time[i] = waiting_time[i] + burst_time[i];
    }
}
```

Shortest Job First (SJF)

c

```
void sjf_scheduling(int processes[], int n, int burst_time[]) {
    // Sort processes by burst time
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (burst_time[j] > burst_time[j+1]) {
                // Swap burst times and process IDs
                int temp = burst_time[j];
                burst_time[j] = burst_time[j+1];
                burst_time[j+1] = temp;

                temp = processes[j];
                processes[j] = processes[j+1];
                processes[j+1] = temp;
            }
        }
    }
    // Calculate waiting and turnaround times
}
```

Round Robin Scheduling

```
c
```

```
void round_robin(int processes[], int n, int burst_time[], int quantum) {
  int remaining_time[n];
  int waiting_time[n] = {0};
  int time = 0;

  // Copy burst times to remaining times
  for (int i = 0; i < n; i++)
    remaining_time[i] = burst_time[i];

  while (1) {
    int done = 1;
    for (int i = 0; i < n; i++) {
      if (remaining_time[i] > 0) {
        done = 0;
        if (remaining_time[i] > quantum) {
          time += quantum;
          remaining_time[i] -= quantum;
        } else {
          time += remaining_time[i];
          waiting_time[i] = time - burst_time[i];
          remaining_time[i] = 0;
        }
      }
    }
    if (done) break;
  }
}
```

6. Synchronization

Mutex (Mutual Exclusion)

```
c
```

```c
#include <pthread.h>

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int shared_variable = 0;

void* thread_function(void* arg) {
    pthread_mutex_lock(&mutex);
    // Critical section
    shared_variable++;
    printf("Shared variable: %d\n", shared_variable);
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

Semaphores

```c
#include <semaphore.h>

sem_t semaphore;

// Initialize semaphore
sem_init(&semaphore, 0, 1); // Binary semaphore

void* producer(void* arg) {
    sem_wait(&semaphore);
    // Critical section
    printf("Producer working\n");
    sem_post(&semaphore);
    return NULL;
}

void* consumer(void* arg) {
    sem_wait(&semaphore);
    // Critical section
    printf("Consumer working\n");
    sem_post(&semaphore);
    return NULL;
}
```

Producer-Consumer Problem

```c
```

```c
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
  int item = 1;
  while (1) {
    sem_wait(&empty);
    pthread_mutex_lock(&mutex);

    buffer[in] = item++;
    in = (in + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&full);
  }
}

void* consumer(void* arg) {
  int item;
  while (1) {
    sem_wait(&full);
    pthread_mutex_lock(&mutex);

    item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);

    printf("Consumed: %d\n", item);
  }
}
```

7. Inter Process Communication (IPC)

Shared Memory

```c
```

```c
#include <sys/ipc.h>
#include <sys/shm.h>

// Create shared memory
key_t key = ftok("shmfile", 65);
int shmid = shmget(key, 1024, 0666 | IPC_CREAT);

// Attach to shared memory
char* shared_memory = (char*) shmat(shmid, (void*)0, 0);

// Write to shared memory
strcpy(shared_memory, "Hello from shared memory");

// Detach from shared memory
shmdt(shared_memory);

// Destroy shared memory
shmctl(shmid, IPC_RMID, NULL);
```

8. Banker's Algorithm (Deadlock Detection)

```c
c
```

```c
#include <stdio.h>

int is_safe(int processes, int resources, int max[][10],
        int allocation[][10], int available[]) {
    int need[10][10];
    int work[10];
    int finish[10] = {0};
    int safe_sequence[10];
    int count = 0;

    // Calculate need matrix
    for (int i = 0; i < processes; i++) {
        for (int j = 0; j < resources; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Initialize work array
    for (int i = 0; i < resources; i++) {
        work[i] = available[i];
    }

    // Find safe sequence
    while (count < processes) {
        int found = 0;
        for (int p = 0; p < processes; p++) {
            if (!finish[p]) {
                int can_allocate = 1;
                for (int j = 0; j < resources; j++) {
                    if (need[p][j] > work[j]) {
                        can_allocate = 0;
                        break;
                    }
                }

                if (can_allocate) {
                    for (int k = 0; k < resources; k++) {
                        work[k] += allocation[p][k];
                    }
                    safe_sequence[count++] = p;
                    finish[p] = 1;
                    found = 1;
                }
            }
        }
```

```
        if (!found) {
            return 0; // System is not in safe state
        }
    }

    return 1; // System is in safe state
}
```

9. Memory Management

Fixed Partitioning

```
c
```

```c
#include <stdio.h>

struct Partition {
    int size;
    int allocated;
    int process_id;
};

void first_fit(struct Partition partitions[], int n, int process_size, int pid) {
    for (int i = 0; i < n; i++) {
        if (!partitions[i].allocated && partitions[i].size >= process_size) {
            partitions[i].allocated = 1;
            partitions[i].process_id = pid;
            printf("Process %d allocated to partition %d\n", pid, i);
            return;
        }
    }
    printf("Process %d could not be allocated\n", pid);
}

void best_fit(struct Partition partitions[], int n, int process_size, int pid) {
    int best_idx = -1;
    int best_size = INT_MAX;

    for (int i = 0; i < n; i++) {
        if (!partitions[i].allocated &&
            partitions[i].size >= process_size &&
            partitions[i].size < best_size) {
            best_size = partitions[i].size;
            best_idx = i;
        }
    }

    if (best_idx != -1) {
        partitions[best_idx].allocated = 1;
        partitions[best_idx].process_id = pid;
        printf("Process %d allocated to partition %d\n", pid, best_idx);
    } else {
        printf("Process %d could not be allocated\n", pid);
    }
}
```

Paging Implementation

```c
```

```c
#include <stdio.h>

#define PAGE_SIZE 4096
#define NUM_PAGES 16
#define NUM_FRAMES 8

struct PageTable {
  int frame_number;
  int valid;
};

int translate_address(struct PageTable page_table[], int logical_address) {
  int page_number = logical_address / PAGE_SIZE;
  int offset = logical_address % PAGE_SIZE;

  if (page_table[page_number].valid) {
    int physical_address = page_table[page_number].frame_number * PAGE_SIZE + offset;
    return physical_address;
  } else {
    printf("Page fault for page %d\n", page_number);
    return -1;
  }
}
```

10. File and Directory Management

File I/O Operations

```c
c
```

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

// File operations
int fd = open("file.txt", O_RDWR | O_CREAT, 0644);
if (fd == -1) {
    perror("open");
    return -1;
}

char buffer[1024];
ssize_t bytes_read = read(fd, buffer, sizeof(buffer));
ssize_t bytes_written = write(fd, "Hello", 5);

close(fd);
```

Directory Management

```bash
# Basic directory operations
mkdir new_directory        # Create directory
rmdir empty_directory      # Remove empty directory
rm -rf directory           # Remove directory and contents
find /path -name "*.txt"   # Find files by pattern
du -sh directory           # Show directory size
```

11. Advanced Bash Scripting

Functions and Parameters

```bash
```

```bash
#!/bin/bash

# Function definition
greet() {
  local name=$1
  local age=$2
  echo "Hello $name, you are $age years old"
}

# Function call
greet "Alice" 25

# Command line arguments
echo "Script name: $0"
echo "First argument: $1"
echo "All arguments: $@"
echo "Number of arguments: $#"
```

File Processing

bash

```bash
#!/bin/bash

# Read file line by line
while IFS= read -r line; do
  echo "Line: $line"
done < "input.txt"

# Process command output
for file in $(ls *.txt); do
  echo "Processing $file"
  wc -l "$file"
done
```

Error Handling

bash

```bash
#!/bin/bash

# Exit on error
set -e

# Check if file exists
if [ ! -f "file.txt" ]; then
    echo "Error: file.txt not found"
    exit 1
fi

# Trap signals
cleanup() {
    echo "Cleaning up..."
    rm -f temp_file
}
trap cleanup EXIT
```

Compilation Commands

Compile C programs with system calls

```bash
# Basic compilation
gcc -o program program.c

# With threading support
gcc -pthread -o program program.c

# With debugging info
gcc -g -o program program.c

# With all warnings
gcc -Wall -Wextra -o program program.c
```

Make executable and run

```bash
chmod +x script.sh
./script.sh

# Or run directly
bash script.sh
```

This cheat sheet covers the essential syntax and concepts for Unix/Linux system programming and operating system fundamentals. Each section includes practical examples that you can compile and run to understand the concepts better.